GOLD-001 — Batch 001
Independent Verification Findings

Production RAG v1 · evidence-candidate authoring · dual-LLM review with mandatory human approval

STATUS — NO CASE IN BATCH 001 IS GOLD

Independent verification is finished. Human review is not. **17 of 18** candidates are queued for a person, and no script in this repository can raise a case to `human_verified`. That status exists only when the owner records an approval.

2 / 2
structural table-row candidates passed with nothing rewritten

9 / 16
prose candidates carrying a named miner defect

15 / 1 / 2
FIX_REQUIRED / FAIL / PASS
UNCERTAIN: 0

17
queued for human review
16 mandatory + 1 QC sample

1. What was run

step	artifact / result
Batch shipped	`gold_review_batch_001.json` — `batch_sha256 a3fe9242e33fb2df…`
Independent review	reviewer `chatgpt` — declared source hash matched exactly, all 18 ids aligned
Import	18 verdicts, 16 versioned revisions, 0 anchor disputes
Human queue	16 mandatory + 1 sampled from the 2 agreed passes (seed 20250819, rate 15%)
Review packet	17 candidates rendered with evidence, context and both proposals

Statuses after import:

status	cases
`needs_human_review`	15
`dual_llm_pass`	2
`dual_llm_fail`	1

THE RULE THE PIPELINE EXISTS TO ENFORCE

A ChatGPT `PASS` produces `dual_llm_pass`, never `human_verified`. Two models agreeing is correlated evidence, not independent confirmation. The distinction lives in code and has a test; it is not left to discipline.

2. What the verdict counts do and do not mean

The raw counts overstate the miner's error rate. Saying so is not a defence of the miner — it is a statement about how the batch was built.

All 16 prose candidates shipped with `[REVIEWER TO WRITE]` as their question and an empty claim list, because GOLD-001 deliberately converted the generator from a question-answer producer into an *evidence-candidate* producer. Four of the six review checks — `question_supported`, `answer_supported`, `all_critical_claims_supported`, `natural_question` —

were therefore **false by construction** on every prose candidate. They carry no information about the miner. `FIX_REQUIRED` on a prose candidate means "the reviewer wrote the question", which is the design working, not a defect.

Two checks do test the miner's own output: whether the anchored span is self-contained, and whether the right identifier was bound to the right relation.

signal	n	candidates
Boundary incomplete	3	03, 04, 14
Identifier / relation binding wrong	6	08, 09, 12, 15, 16, 17
Either defect	9	of 16 prose candidates
No named miner defect — needed only a question written	7	05, 06, 07, 10, 11, 13, 18

> **HONESTY ABOUT N**
>
> With n = 18, one candidate is 5.6 percentage points. The structural miner went 2 for 2 — that is two observations, not a precision estimate, and it is reported as two observations. No significance claim is made anywhere in this document.

3. Defect taxonomy

D1 — anaphoric anchor (3 cases)

The span opens with, or silently depends on, a referent that lives outside it. The reviewer cannot check the claim against the anchor alone, which is the whole contract.

> `"If true, an [InputGuardrailTripwireTriggered] exception is raised…"` — *what is true is named only in the preceding sentence*

> `"Sending a request with a prefilled last assistant message to **any of these models** returns a 400 invalid_request_error:"` — *"these models" is the question, and is not in the span*

This is the same shape as the OA-002 defect already recorded against the original human-verified set. Finding it three times in 16 candidates means the miner reproduces it **systematically**, not by accident.

D2 — wrong relation label (5 cases)

The miner matched a trigger word and labelled the candidate with a relation the sentence does not express. The evidence is usually fine; the *label* aims the reviewer at the wrong question — one case linked `file_id` to wording from a different bullet in the same error list; another labelled a two-part stopping condition as a single response relation. Because the label rode along in the exported candidate, it steered the reviewer's first reading. On this batch's evidence, the label is worse than no label.

D3 — identifier matched inside example code (2 cases, including the only FAIL)

The miner matched `"required": ["location"]` inside a JSON request body and framed it as a requirement on `tool_choice`.

> **THIS IS THE EXP-014R FAILURE MODE, REPRODUCED**
>
> False token-to-identifier association is the exact defect that made the EXP-014R generator unusable and the reason GOLD-001 was commissioned. It survived into a shipped batch. A sample configuration is not a documented rule, and the miner currently cannot tell them apart.

4. Changes preregistered for batch 002

Written down **before** batch 002 is generated, so they cannot be tuned to its outcome. None of them touch batch 001; its records stay exactly as verified.

1. **Reject or extend anaphoric spans.** A span opening with `If true`, `these`, `it`, `this`, or whose subject is a bare pronoun, is extended to include its antecedent or dropped. → D1
2. **Stop exporting the proposed relation label.** Keep it internally for selection; remove it from what the reviewer sees. Wrong on 5 of 16 and misleading when wrong. → D2
3. **Refuse identifier matches inside fenced code blocks and JSON literals** when the candidate would be framed as a documented rule. → D3
4. **Raise the share of structural candidates.** Table-row mining produced the only candidates needing no re-authoring. This is a change of mix, not a claim that table rows have high precision in general.

5. What is unchanged

invariant	state
Retrieval run over any candidate	**never — `retrieval_was_not_run: true`**
SYSTEM-A / SYSTEM-B	**frozen at `9afcb5b7…` / `304c3509…`, not run against any candidate**
Holdout	**not frozen; no A-vs-B replication attempted**
OA-002	recorded defect, correction proposed and unapplied, awaiting the owner's decision
EXP-NULL	BLOCKED — no project generation credential

6. Next step, and who owns it

> **THE NEXT STEP IS NOT BATCH 002**
>
> It is the human review of the 17 queued candidates, because that is the only step that can produce a `human_verified` case. The target of 30–40 validation plus 70–100 holdout cases is gated entirely on it — more candidates are not the constraint.
>
> Regenerate the packet with `scripts/export_human_qc_packet.py`, then record `APPROVE` or `REJECT` per candidate in `evals/review/human_decisions_batch_001.json`.

Generated from evals/review/*.json by scripts/build_gold001_pdf.py — every count in this document is read from those artifacts at build time. Batch 001, schema 1.0, git commit 67de5563f469, corpus snapshot snap_689e336380a054d8039dc35b2c09cd0a. Reviewed by chatgpt on 2026-08-19T05:41:57Z. Nothing in batch 001 is gold. Raw provider documentation is not redistributed; quoted spans are the short excerpts needed to show each defect.